# task3_rating_classification

October 26, 2026

## 1 Task 3: Netflix Audience Rating Classification

**Auspify Technologies – Machine Learning Internship**

Goal: predict the audience rating category of a Netflix title (Kids, Older Kids, Teens, Adults) from its content attributes. This is a multi-class classification problem.

[ ]:
```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score,␣
↪StratifiedKFold, GridSearchCV, RandomizedSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, f1_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

sns.set_theme(style="whitegrid")
RS = 42
df = pd.read_csv("Dataset.csv")
print(df.shape)
df.head()
```

### 1.1 1. Analyze rating categories

Netflix uses many rating labels (TV-MA, TV-14, R, PG-13…). We are to look at how they are distributed.

[ ]:
```
print(df["rating"].value_counts(dropna=False))
df["rating"].value_counts().sort_values().plot.barh(figsize=(7, 5),␣
↪color="#e50914")
```

```
plt.title("Distribution of raw rating labels"); plt.xlabel("Number of titles");␣
 ↪plt.show()
```

Some labels are very rare (e.g. NC-17, TV-Y7-FV), and a few titles have no usable rating (NR, UR, missing). With so few examples a model cannot learn them. So the raw labels are grouped into **four audience categories** that follow the age level of the viewer:

| Category | Ratings |
|---|---|
| **Kids** | TV-Y, TV-G, G |
| **Older Kids** | TV-Y7, TV-Y7-FV, TV-PG, PG |
| **Teens** | TV-14, PG-13 |
| **Adults** | TV-MA, R, NC-17 |

Titles with an unrated or missing label (NR, UR, NaN) are removed because their audience is unknown.

[ ]:
```
rating_map = {
    "TV-Y": "Kids", "TV-G": "Kids", "G": "Kids",
    "TV-Y7": "Older Kids", "TV-Y7-FV": "Older Kids", "TV-PG": "Older Kids",␣
 ↪"PG": "Older Kids",
    "TV-14": "Teens", "PG-13": "Teens",
    "TV-MA": "Adults", "R": "Adults", "NC-17": "Adults",
}
ORDER = ["Kids", "Older Kids", "Teens", "Adults"]

data = df.drop_duplicates(subset="title").copy()
data["audience"] = data["rating"].map(rating_map)
print("Rows removed (unrated / unknown rating):", data["audience"].isna().sum())
data = data.dropna(subset=["audience"]).reset_index(drop=True)

share = data["audience"].value_counts(normalize=True).reindex(ORDER)
print(share.round(3))
share.plot.bar(color=["#b3b3b3", "#b3b3b3", "#221f1f", "#e50914"], rot=0,␣
 ↪figsize=(6, 3.5))
plt.title("Audience categories"); plt.ylabel("Share of titles"); plt.show()
```

The classes are **imbalanced**: For that we use `class_weight="balanced"` during training to avoid model predicting always the bigest class.

## 1.2 2. Prepare the training dataset

Features built from the content attributes: - **type** (Movie / TV Show) - **duration**: minutes for movies, number of seasons for TV shows - **release_year**, **year/month added** to Netflix, and the **delay** between release and addition - **main country** (first country listed; rare countries are grouped) - **director known** (0/1) and **number of genres** - **genres** (multi-hot encoding of `listed_in`: one 0/1 column per genre)

`rating` is the target.

```
[ ]: # Dates
d = pd.to_datetime(data["date_added"], format="%m/%d/%Y", errors="coerce")
data["added_year"] = d.dt.year.fillna(d.dt.year.median())
data["added_month"] = d.dt.month.fillna(d.dt.month.median())
data["years_to_add"] = data["added_year"] - data["release_year"]

# Duration: minutes for movies, seasons for TV shows
num = data["duration"].str.extract(r"(\d+)")[0].astype(float)
is_movie = data["type"] == "Movie"
data["movie_minutes"] = np.where(is_movie, num, 0).astype(float)
data["n_seasons"] = np.where(~is_movie, num, 0).astype(float)
data["movie_minutes"] = data["movie_minutes"].fillna(0)
data["n_seasons"] = data["n_seasons"].fillna(0)

# Other simple features
data["main_country"] = data["country"].fillna("Unknown").str.split(",").str[0].
 ↪str.strip()
data["director_known"] = (~data["director"].fillna("Not Given").isin(["Not␣
 ↪Given", ""])).astype(int)
data["n_genres"] = data["listed_in"].str.count(",") + 1

# Genres: multi-hot, keeping only genres with at least 30 titles
lists = data["listed_in"].str.split(",").apply(lambda xs: [x.strip() for x in␣
 ↪xs])
genres = pd.get_dummies(lists.explode()).groupby(level=0).max().astype(int)
genres = genres.loc[:, genres.sum() >= 30]
genres.columns = [f"genre_{c}" for c in genres.columns]
data = pd.concat([data, genres], axis=1)

NUM = ["release_year", "added_year", "added_month", "years_to_add",␣
 ↪"movie_minutes", "n_seasons", "n_genres"]
CAT = ["type", "main_country"]
BIN = ["director_known"] + list(genres.columns)

X = data[NUM + CAT + BIN]
y = data["audience"]
print("Features:", X.shape[1], "| Titles:", len(X), "| Genre columns:", genres.
 ↪shape[1])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪stratify=y, random_state=RS)
print("Train:", X_train.shape, "| Test:", X_test.shape)
```

**Encoding** is done inside a scikit-learn `Pipeline`: categorical columns use One-Hot encoding (rare countries are grouped), numeric columns are standardized, and genres are passed through as 0/1. Because everything lives in the pipeline, the encoders are fitted **only on the training data**, so no information leaks from the test set.

The split is 80 % train / 20 % test, **stratified** so each class keeps the same proportion in both sets.

[ ]:
```
def make_pipeline(model):
    prep = ColumnTransformer([
        ("num", StandardScaler(), NUM),
        ("cat", OneHotEncoder(handle_unknown="infrequent_if_exist",␣
 ↪min_frequency=30), CAT),
        ("bin", "passthrough", BIN),
    ])
    return Pipeline([("prep", prep), ("model", model)])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RS)
```

### 1.3 3. Train classification models

We compare four models with default-ish settings: - **Baseline**: always predicts the most frequent class - **Logistic Regression**: simple linear reference - **Decision Tree** (`max_depth=8` so it does not simply memorize the data) - **Random Forest**: many trees averaged together

[ ]:
```
models = {
    "Baseline (most frequent)": DummyClassifier(strategy="most_frequent"),
    "Logistic Regression":      LogisticRegression(max_iter=3000,␣
 ↪class_weight="balanced"),
    "Decision Tree":            DecisionTreeClassifier(max_depth=8,␣
 ↪class_weight="balanced", random_state=RS),
    "Random Forest":            RandomForestClassifier(n_estimators=300,␣
 ↪class_weight="balanced", random_state=RS, n_jobs=-1),
}
fitted = {name: make_pipeline(m).fit(X_train, y_train) for name, m in models.
 ↪items()}
print("Training finished:", list(fitted))
```

### 1.4 4. Optimize model performance (hyperparameter tuning)

We tune the two tree-based models with cross-validation on the **training set only**, optimizing **macro F1**: - Decision Tree: `GridSearchCV` (small search space, tested exhaustively) - Random Forest: `RandomizedSearchCV` (larger search space, 15 random combinations)

[ ]:
```
# Decision Tree: grid search
dt_grid = {
    "model__criterion": ["gini", "entropy"],
    "model__max_depth": [4, 6, 8, 10, 14, None],
    "model__min_samples_leaf": [1, 5, 10, 20],
}
dt_search = GridSearchCV(
    make_pipeline(DecisionTreeClassifier(class_weight="balanced",␣
 ↪random_state=RS)),
    dt_grid, scoring="f1_macro", cv=cv, n_jobs=-1)
```

```
dt_search.fit(X_train, y_train)
print("Best Decision Tree params:", dt_search.best_params_)
print("Best CV macro F1:", round(dt_search.best_score_, 4))
```

[ ]:
```
# Random Forest: randomized search
rf_dist = {
    "model__n_estimators": [200, 300, 500],
    "model__max_depth": [8, 12, 16, 20, None],
    "model__min_samples_leaf": [1, 2, 5, 10],
    "model__max_features": ["sqrt", "log2", 0.3],
}
rf_search = RandomizedSearchCV(
    make_pipeline(RandomForestClassifier(class_weight="balanced",␣
 ↪random_state=RS, n_jobs=-1)),
    rf_dist, n_iter=15, scoring="f1_macro", cv=cv, random_state=RS, n_jobs=1)
rf_search.fit(X_train, y_train)
print("Best Random Forest params:", rf_search.best_params_)
print("Best CV macro F1:", round(rf_search.best_score_, 4))
```

[ ]:
```
fitted["Decision Tree (tuned)"] = dt_search.best_estimator_
fitted["Random Forest (tuned)"] = rf_search.best_estimator_
```

### 1.5 5. Evaluate prediction accuracy and compare models

All models are scored on the **held-out test set** (never seen during training or tuning). Metrics: - **Accuracy**: share of correct predictions - **Macro F1**: F1 averaged over the 4 classes (treats small classes fairly) - **Weighted F1**: F1 averaged by class size

[ ]:
```
rows = []
for name, pipe in fitted.items():
    pred = pipe.predict(X_test)
    rows.append({
        "model": name,
        "accuracy": accuracy_score(y_test, pred),
        "macro F1": f1_score(y_test, pred, average="macro"),
        "weighted F1": f1_score(y_test, pred, average="weighted"),
    })
res = pd.DataFrame(rows).set_index("model").round(3)
res
```

[ ]:
```
ax = res[["accuracy", "macro F1", "weighted F1"]].plot.bar(figsize=(11, 4.5),␣
 ↪rot=20,
                                                           color=["#e50914",␣
 ↪"#221f1f", "#b3b3b3"])
ax.set_ylim(0, 1); ax.set_title("Model comparison on the test set"); ax.
 ↪legend(loc="lower right")
plt.tight_layout(); plt.show()
```

```
best = res["macro F1"].idxmax()
print("Best model (macro F1):", best)
pred = fitted[best].predict(X_test)
print(classification_report(y_test, pred, labels=ORDER, digits=3))

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, labels=ORDER,␣
 ↪cmap="Reds", ax=axes[0], colorbar=False)
axes[0].set_title(f"Confusion matrix (counts): {best}")
ConfusionMatrixDisplay.from_predictions(y_test, pred, labels=ORDER,␣
 ↪normalize="true", values_format=".2f",
                                        cmap="Reds", ax=axes[1], colorbar=False)
axes[1].set_title("Confusion matrix (share of each true class)")
plt.tight_layout(); plt.show()
```

Reading the confusion matrix: each row is the true category, each column the predicted one. Mistakes between **neighbouring** categories (for example Teens vs Adults) are expected and less serious than mistakes between distant ones (Kids vs Adults).

### 1.5.1 Which features matter most?

```
rf = fitted["Random Forest (tuned)"]
names = rf.named_steps["prep"].get_feature_names_out()
imp = pd.Series(rf.named_steps["model"].feature_importances_, index=names).
 ↪sort_values(ascending=False).head(15)
imp.index = [i.split("__", 1)[1] for i in imp.index]
imp.sort_values().plot.barh(figsize=(8, 5), color="#e50914")
plt.title("Top 15 features (tuned Random Forest)"); plt.show()
```

```
base = res.loc["Baseline (most frequent)", "accuracy"]
print(f"Baseline accuracy : {base:.3f}")
print(f"Best model        : {best}")
print(f"  accuracy        : {res.loc[best, 'accuracy']:.3f}  (+{res.loc[best,␣
 ↪'accuracy'] - base:.3f} over baseline)")
print(f"  macro F1        : {res.loc[best, 'macro F1']:.3f}")
tuned_gain = res.loc["Random Forest (tuned)", "macro F1"] - res.loc["Random␣
 ↪Forest", "macro F1"]
print(f"Tuning effect on Random Forest macro F1: {tuned_gain:+.3f}")
```

## 1.6 Conclusion

- **Target:** the 12 raw rating labels were grouped into 4 audience categories (Kids, Older Kids, Teens, Adults) because several labels were too rare to learn. Unrated titles were removed.
- **Imbalance:** Adults and Teens make up most titles, so a "most frequent class" baseline already scores well. For that reason the comparison relies on **macro F1** and per-class results, not accuracy alone.
- **Models:** Decision Tree and Random Forest (plus Logistic Regression) were compared with the baseline. Hyperparameter tuning used cross-validation on the training set only, so the

test scores are an honest estimate. The table in Step 5 shows the final numbers and which model wins; the tuning effect is printed in the last code cell.

- **Features:** the most informative features are shown in the importance chart above. Genres (such as children's content, stand-up, or documentaries) and the title type usually carry most of the signal.
- **Limitations:** the dataset has no synopsis, cast, or content descriptors, which are what really determine a rating (violence, language…). Confusion between neighbouring categories (Teens vs Adults) is therefore hard to remove with these attributes alone.
- **perspectives:** add text features (TF-IDF on title/description if available), try gradient boosting, and test a finer 6-class grouping.

[ ]: